Malaria Cell Classification System
A Concise MLOps Project Report

Keerthana Sridhar

April 28, 2026

Contents

1 Introduction **3**

2 Project Objective and Scope **3**

3 Dataset Overview **3**

4 Repository Overview **4**

5 Repository Organisation **4**

5.1 Technology Stack 5

6 System Architecture **5**

6.1 High-Level Architecture 5

6.2 Containerized Deployment Architecture 6

7 System Design **7**

7.1 Overall Architecture 7

7.2 Containerized Deployment 7

8 User-Facing Application **8**

8.1 Home Page and User Experience 8

8.2 Sidebar Metrics and Failure Visibility 10

8.3 Experiments and Model Metadata 11

9 Data Pipeline and Reproducibility **11**

9.1 Dataset Summary and EDA 11
9.2 DVC-Managed Pipeline 11

10 Model Development and Evaluation **12**
10.1 Model Choice 12
10.2 Training and Evaluation Results 12

11 MLflow Experiment Tracking and Model Registry **13**

12 Inference, Alerts, and Monitoring **14**
12.1 Runtime Monitoring in the UI 14
12.2 Prometheus and Alertmanager 14
12.3 Grafana Dashboards 15

13 Human-in-the-Loop Feedback and Retraining **15**

14 Backend API Design **17**

15 Testing and Software Engineering Quality **17**

16 Conclusion **18**

A AI Disclosure **18**

Abstract

This report presents an end-to-end malaria cell classification system built as an MLOps application rather than as a standalone model. The project combines a Streamlit frontend, a FastAPI backend, a DVC-based reproducible machine learning pipeline, MLflow experiment tracking and model registry, Airflow-based incremental retraining, and a Prometheus–Grafana–Alertmanager monitoring stack. The focus of the system is twofold: first, to provide a usable interface for image-based malaria classification; and second, to demonstrate a practical workflow for training, deploying, observing, and continuously improving a model in a controlled and reproducible way. Throughout the report, screenshots are used as evidence to connect the implemented repository with the expectations in the MLOps build guideline and the project evaluation rubric.

1 Introduction

This project was developed as a complete AI application for malaria cell image classification. Instead of stopping at model training, the repository was organized to reflect the full lifecycle of a real ML system: data preparation, experiment tracking, model serving, monitoring, feedback collection, retraining, and deployment packaging. That broader framing is important because the evaluation guidelines emphasize not only prediction quality, but also usability, reproducibility, traceability, and operational maturity.

The application supports both technical and non-technical use. A clinician or evaluator can upload cell images through a web interface and receive predictions, while the engineering side of the system keeps track of model versions, runtime health, drift signals, feedback queues, and retraining activity. This makes the project a good demonstration of how MLOps tools can be used to turn an ML model into a maintainable product.

2 Project Objective and Scope

The main goal of the project is to classify malaria cell images into two categories: *Parasitized* and *Uninfected*. To make that goal practical and sustainable, the repository was built around a broader set of objectives. The system needed to provide a frontend for inference, backend APIs for prediction and operations, a reproducible DVC workflow for training and evaluation, MLflow-based tracking and model versioning, Airflow-based retraining for newly labelled data, and live monitoring through Prometheus and Grafana.

3 Dataset Overview

The project uses the Cell Images for Detecting Malaria dataset, which is a publicly available collection of microscopic images of red blood cells. These images were captured using thin blood smear slides and are commonly used in malaria diagnosis workflows. The dataset consists of two balanced classes: Parasitized: Images containing red blood cells infected with malaria

parasites. Uninfected: Images of healthy red blood cells without infection. In total, the dataset contains 27,558 images, evenly split between the two categories, making it suitable for supervised classification tasks without severe class imbalance issues.

Each image is: A colored microscopic image Typically resized to a fixed resolution (commonly 64×64 or 128×128 in preprocessing pipelines) Stored in a directory structure where folder names correspond to class labels

link: https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria

4 Repository Overview

The repository is structured as a modular application. The `src/` folder contains the backend logic, training scripts, evaluation scripts, feedback processing, and MLflow utilities. The `frontend/` folder contains the Streamlit user interface. The `airflow/` folder contains the retraining DAG and Airflow configuration. The `docker/` directory contains Dockerfiles and entrypoints for containerized deployment. The `monitoring/` folder contains Grafana and Alertmanager configuration. Finally, `docs/` contains the architecture, high-level design, low-level design, test plan, user manual, acceptance criteria, and reproducibility notes.

This layout already reflects one of the stronger software-engineering aspects of the project: the code is not mixed into a single notebook or script. Instead, the responsibilities are separated in a way that maps naturally onto the system architecture.

5 Repository Organisation

The repository is organized as an AI application codebase rather than as a single notebook or isolated model training script. The key top-level components are:

- `src/`: backend logic, model training, evaluation, feedback processing, MLflow utilities, and serving logic.
- `frontend/`: Streamlit user interface.
- `airflow/`: Airflow DAGs and containerization for orchestration.
- `docker/`: Dockerfiles and service entrypoints.
- `docs/`: architecture, high-level design, low-level design, test plan, user manual, and reproducibility notes.
- `monitoring/`: Grafana provisioning and Alertmanager configuration.
- `dvc.yaml` and `dvc.lock`: reproducible ML pipeline definition.
- `docker-compose.yaml`: multi-service deployment definition.

5.1 Technology Stack

The current repository uses the following tools and frameworks:

- Frontend: Streamlit
- Backend APIs: FastAPI
- Deep Learning: PyTorch and Torchvision
- Experiment Tracking and Registry: MLflow
- Reproducible Pipeline: DVC
- Workflow Orchestration: Apache Airflow
- Monitoring: Prometheus
- Visualization: Grafana
- Alert Routing: Alertmanager
- Containerization: Docker and Docker Compose

6 System Architecture

The system follows a loosely coupled architecture in which the frontend and backend are separate software blocks connected through REST APIs. This is consistent with the evaluation guideline requirement that the UI and inference engine remain independent.

6.1 High-Level Architecture

At a high level, the system contains the following blocks:

1. **User-facing frontend**: a Streamlit application for uploading images, viewing predictions, inspecting pipeline state, and observing retraining and monitoring signals.
2. **Inference backend**: a FastAPI application that loads the currently served model, executes prediction, collects metrics, and exposes operational APIs.
3. **Model registry and experiment store**: MLflow tracks runs, parameters, metrics, artifacts, and registered models.
4. **Reproducible training pipeline**: DVC defines the complete tracked workflow for EDA, preprocessing, resizing, training, and evaluation.
5. **Operational retraining workflow**: Airflow orchestrates the incremental feedback loop for newly labelled data.

6. **Monitoring stack**: Prometheus scrapes metrics, Grafana visualizes them, and Alertmanager forwards active alerts back to the backend.

Overall system architecture Use the architecture diagram from your documentation or export the Mermaid diagram. Show frontend, backend, MLflow, DVC pipeline, Airflow retraining, Prometheus, Grafana, and Alertmanager in one figure. fig-architecture-overview.png

6.2 Containerized Deployment Architecture

The deployment is defined through `docker-compose.yaml`. The Compose stack includes at least the following important services:

- `frontend`
- `backend`
- `mlflow`
- `prometheus`
- `grafana`
- `alertmanager`
- `postgres`
- `redis`
- `airflow-apiserver`
- `airflow-scheduler`
- `airflow-dag-processor`
- `airflow-worker`
- `airflow-triggerer`

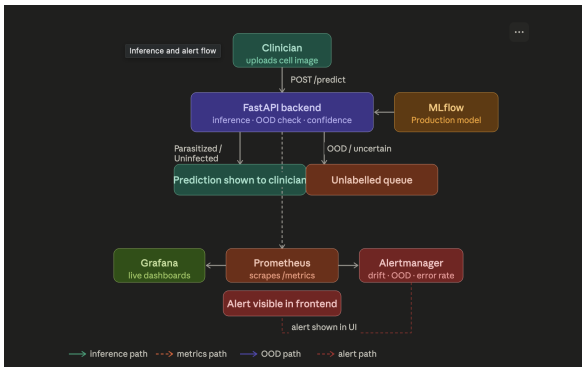
This multi-container setup demonstrates environment parity and operational separation of concerns.

Docker Compose service stack Show terminal output of `docker compose ps` with frontend, backend, mlflow, prometheus, grafana, alertmanager, and Airflow services running. fig-docker-compose-services.png

7 System Design

7.1 Overall Architecture

The system follows a loosely coupled design. The Streamlit frontend talks to the FastAPI backend only through REST calls. The backend handles inference and operational APIs, while MLflow stores experiments and model versions. Prometheus scrapes backend metrics, Grafana visualizes them, and Alertmanager pushes active alerts back into the application flow. Separately, Airflow manages the short feedback-retraining loop, while DVC manages the full reproducible training pipeline.



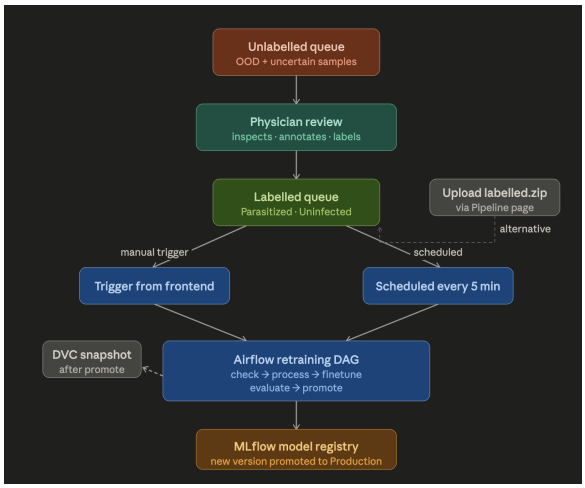
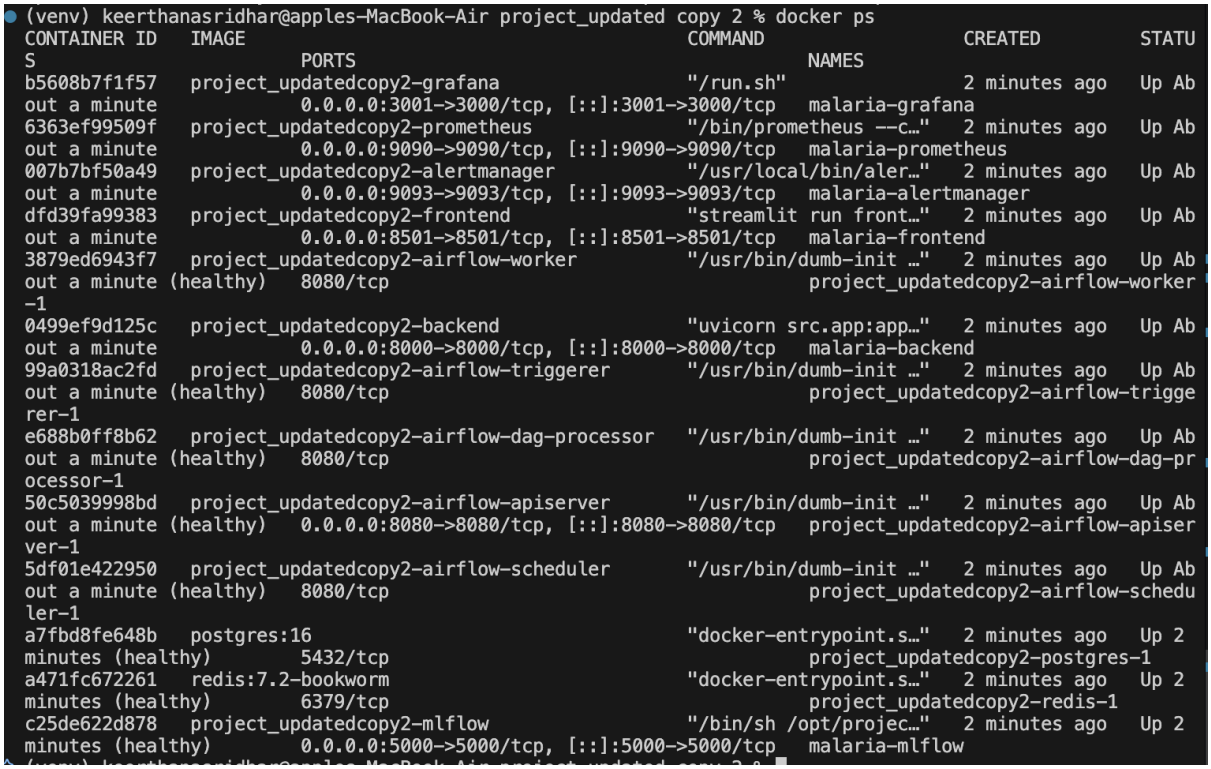


Figure 1: These two diagrams summarize the operating logic of the system. The first image focuses on the inference and alert path: a clinician uploads an image, the backend serves the MLflow Production model, Prometheus scrapes runtime metrics, Grafana visualizes them, and Alertmanager returns important alerts to the UI. The second image shows the human-in-the-loop retraining loop, where uncertain or OOD samples are reviewed by a physician, sent to the labelled queue, and then used by the Airflow DAG to produce a new promoted model version.

From a design point of view, these two diagrams capture the heart of the project. The first is about serving and observation. The second is about learning from new data without manually re-running the entire workflow each time.

7.2 Containerized Deployment

The project is packaged as a multi-container system using Docker Compose. The important services include a frontend, backend, MLflow server, Prometheus, Grafana, Alertmanager, Redis, PostgreSQL for Airflow, and several Airflow components such as the scheduler, worker, API server, triggerer, and DAG processor.



```
(venv) keerthanasridhar@apples-MacBook-Air project_updated copy 2 % docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED         STATU
S                      PORTS                                                   NAMES
b5608b7f1f57   project_updatedcopy2-grafana                   "/run.sh"                2 minutes ago   Up Ab
out a minute           0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp   malaria-grafana
6363ef99509f   project_updatedcopy2-prometheus                "/bin/prometheus --c…"   2 minutes ago   Up Ab
out a minute           0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   malaria-prometheus
007b7bf50a49   project_updatedcopy2-alertmanager              "/usr/local/bin/aler…"   2 minutes ago   Up Ab
out a minute           0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp   malaria-alertmanager
dfd39fa99383   project_updatedcopy2-frontend                  "streamlit run front…"   2 minutes ago   Up Ab
out a minute           0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp   malaria-frontend
3879ed6943f7   project_updatedcopy2-airflow-worker            "/usr/bin/dumb-init …"   2 minutes ago   Up Ab
out a minute (healthy)   8080/tcp                                          project_updatedcopy2-airflow-worker
-1
0499ef9d125c   project_updatedcopy2-backend                   "uvicorn src.app:app…"   2 minutes ago   Up Ab
out a minute           0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   malaria-backend
99a0318ac2fd   project_updatedcopy2-airflow-triggerer         "/usr/bin/dumb-init …"   2 minutes ago   Up Ab
out a minute (healthy)   8080/tcp                                          project_updatedcopy2-airflow-trigge
rer-1
e688b0ff8b62   project_updatedcopy2-airflow-dag-processor     "/usr/bin/dumb-init …"   2 minutes ago   Up Ab
out a minute (healthy)   8080/tcp                                          project_updatedcopy2-airflow-dag-pr
ocessor-1
50c5039998bd   project_updatedcopy2-airflow-apiserver         "/usr/bin/dumb-init …"   2 minutes ago   Up Ab
out a minute (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   project_updatedcopy2-airflow-apiser
ver-1
5df01e422950   project_updatedcopy2-airflow-scheduler         "/usr/bin/dumb-init …"   2 minutes ago   Up Ab
out a minute (healthy)   8080/tcp                                          project_updatedcopy2-airflow-schedu
ler-1
a7fbd8fe648b   postgres:16                                    "docker-entrypoint.s…"   2 minutes ago   Up 2
minutes (healthy)        5432/tcp                                          project_updatedcopy2-postgres-1
a471fc672261   redis:7.2-bookworm                             "docker-entrypoint.s…"   2 minutes ago   Up 2
minutes (healthy)        6379/tcp                                          project_updatedcopy2-redis-1
c25de622d878   project_updatedcopy2-mlflow                    "/bin/sh /opt/projec…"   2 minutes ago   Up 2
minutes (healthy)        0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   malaria-mlflow
```

Figure 2: The Docker process listing confirms that the system is deployed as a multi-service application rather than a single script. The screenshot shows the frontend, backend, MLflow, Prometheus, Grafana, Alertmanager, Redis, PostgreSQL, and multiple Airflow services all running together. This is strong evidence of environment parity and clean separation between serving, monitoring, orchestration, and experimentation components.

This deployment style directly supports the requirement from the guidelines that the environments should be consistent and containerized. It also makes the demo more convincing because the audience can see that the frontend, backend, retraining system, and monitoring stack are genuinely connected services.

8 User-Facing Application

8.1 Home Page and User Experience

The Streamlit frontend is the most visible part of the system for a non-technical evaluator. The Home page is designed to be straightforward: the user sees the application title, backend connectivity state, system alert status, and an upload control for individual images or ZIP files.

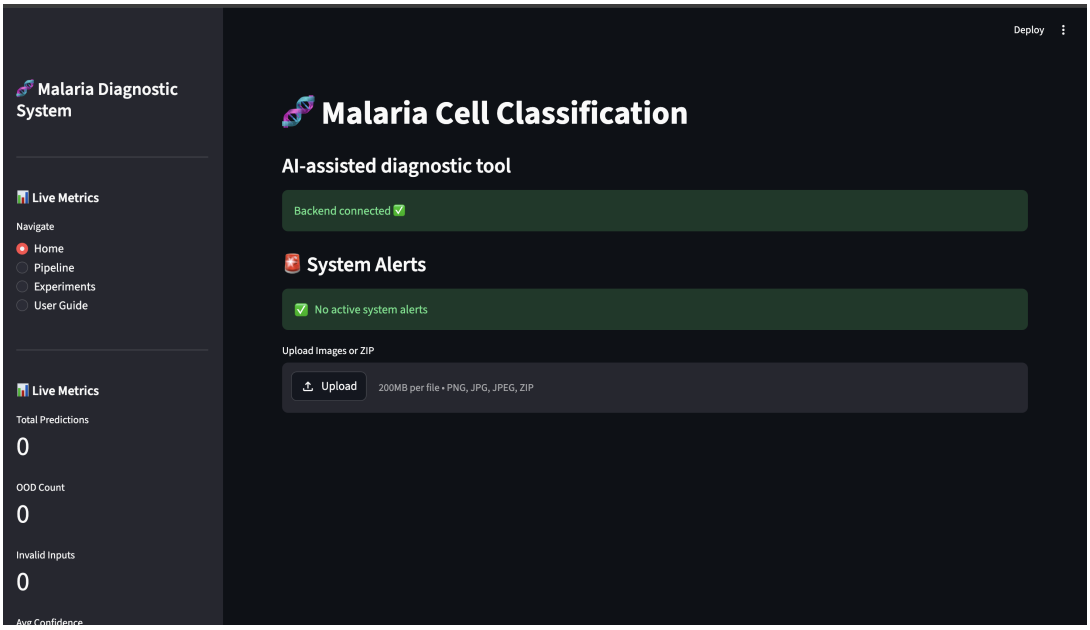


Figure 3: The Home page shows a clean starting point for a non-technical user. It makes the system state visible immediately by showing backend connectivity, active system alerts, and the file upload area.

Once an image is uploaded, the user can preview it and run analysis. The response is presented in a readable table with prediction, confidence, latency, status, and a message field.

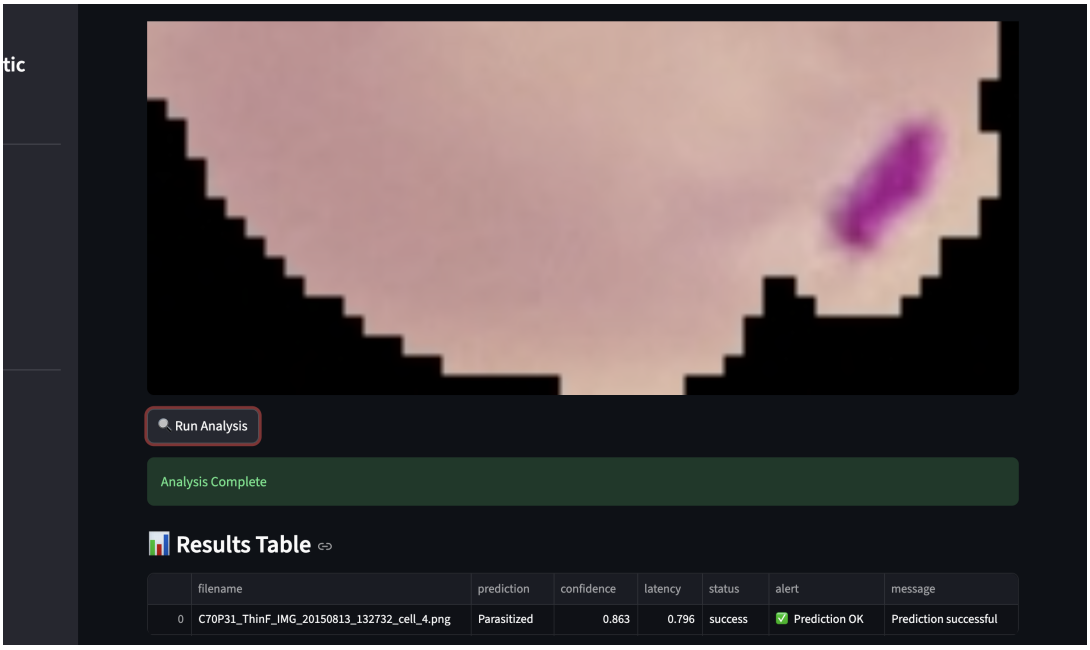


Figure 4: This screenshot shows a successful end-to-end inference from the user's perspective. The uploaded image is visible, the analysis button is clearly exposed, and the results table explains what the model predicted, how confident it was, how long inference took, and whether the overall status was successful. The table format also makes the application easier to demo because all key outputs are visible in one place.

8.2 Sidebar Metrics and Failure Visibility

A useful design choice in the frontend is the sidebar. Instead of hiding runtime information behind a developer console, the application exposes live metrics, OOD counts, invalid input counts, confidence summaries, and a failure log directly in the UI.

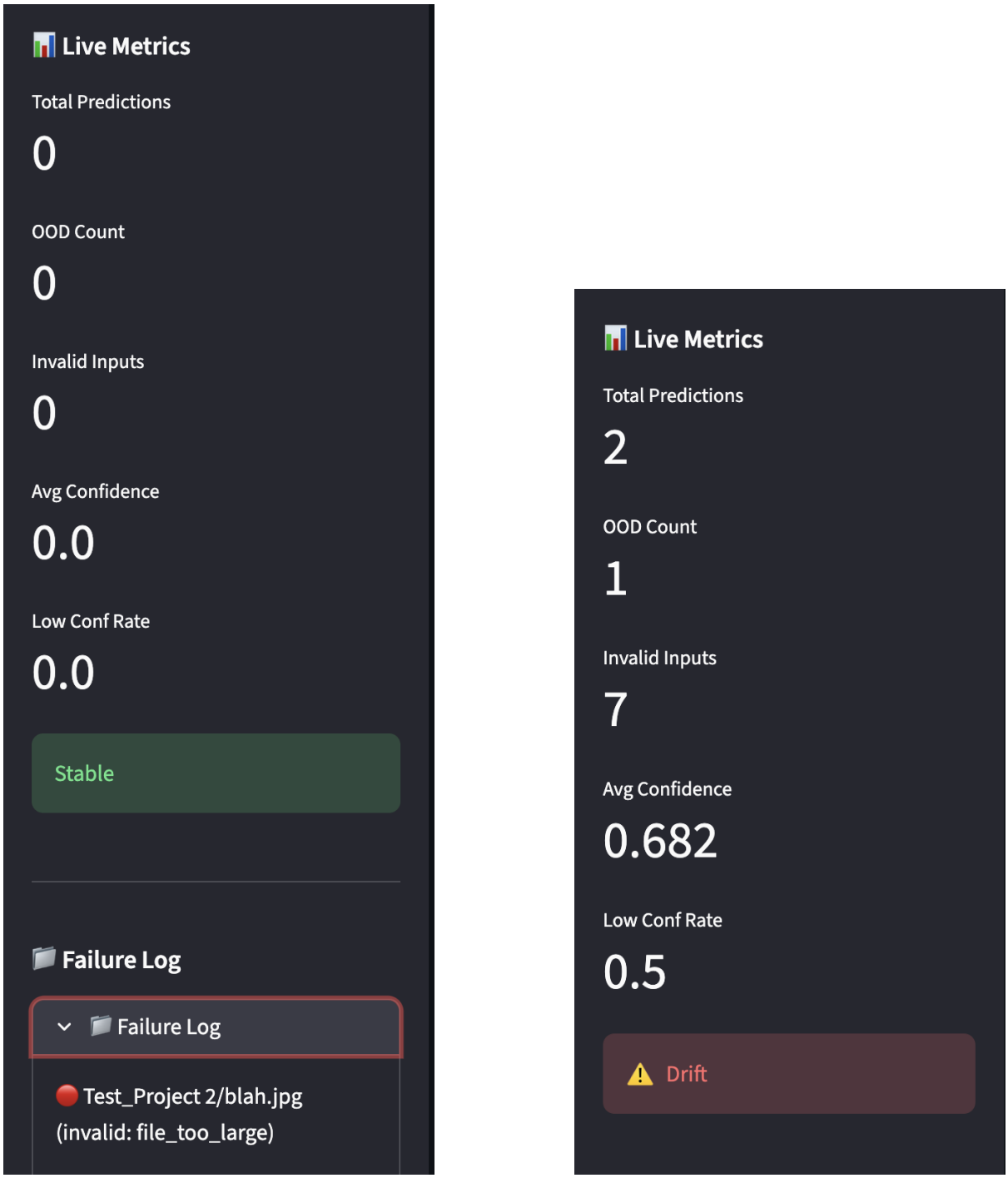


Figure 5: The left image shows the sidebar in a calm, healthy state with zero counts and a stable model status. The right image shows the same interface after real usage, where total predictions, OOD cases, invalid inputs, confidence average, and drift become visible. This is a strong UI decision because it turns operational signals into something an evaluator can understand without opening Prometheus or inspecting raw logs.

8.3 Experiments and Model Metadata

The frontend also includes a dedicated Experiments page that surfaces the currently served model information. This includes the model name, run ID, run name, version, and refresh state.

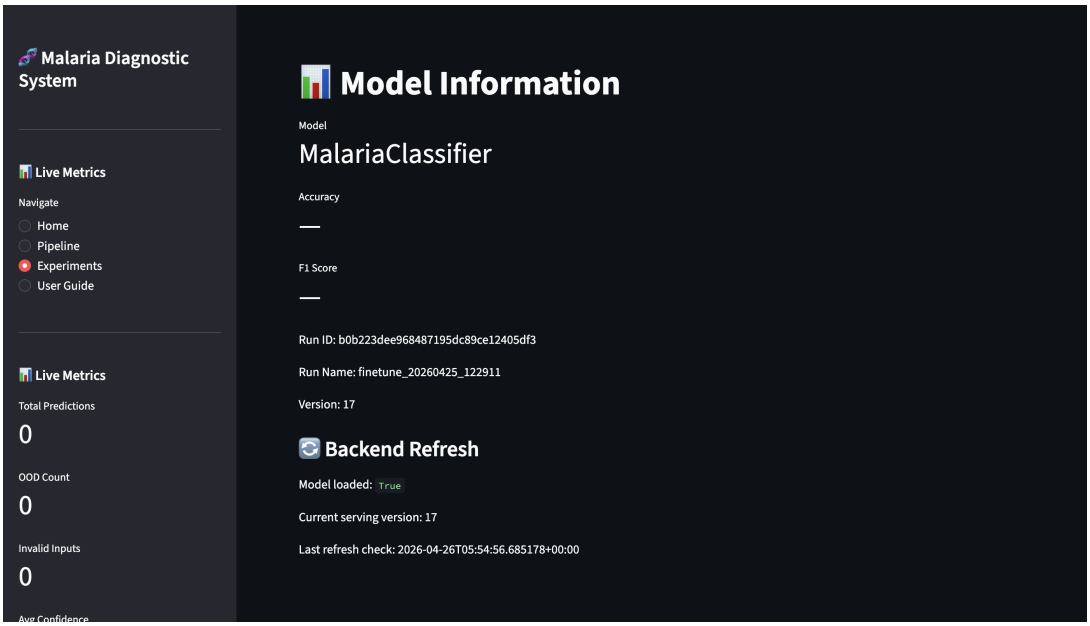


Figure 6: The Experiments page bridges the gap between the UI and the MLOps back end. It shows which model is currently being served, which run produced it, and which model version is active. This makes the model registry visible to the end user and supports reproducibility, because a prediction is no longer detached from its underlying training run and promoted version.

9 Data Pipeline and Reproducibility

9.1 Dataset Summary and EDA

The repository contains a clear EDA stage in `src/eda.py`. It computes class distribution, image size statistics, corrupt image counts, and channel statistics that are later reused for drift and OOD checks. Based on the current generated metrics, the dataset is balanced, with 13,816 images in each class, for a total of 27,632 images. The code also reports average width and height, as well as two corrupt images.

This stage matters for two reasons. First, it gives a quantitative picture of the dataset before training. Second, it provides baseline statistics for later runtime monitoring.

9.2 DVC-Managed Pipeline

The full reproducible workflow is defined in `dvc.yaml`. The stages are `merge_feedback`, `eda`, `preprocess`, `resize`, `train`, and `evaluate`. The preprocessing code splits the raw data into

train, validation, and test sets using the configured ratios 0.70, 0.15, and 0.15. Images are then resized to 224×224.

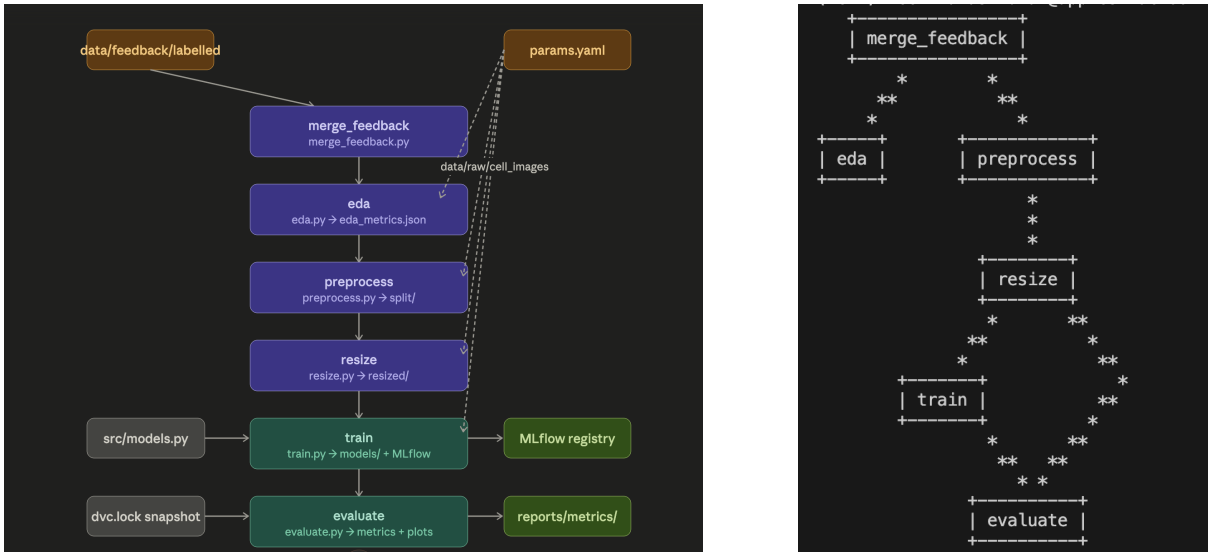


Figure 7: These two images complement each other well. The block diagram on the left explains the logical flow of the reproducible ML pipeline, including feedback merging, EDA, preprocessing, resizing, training, evaluation, and MLflow registration. The DVC DAG screenshot on the right acts as proof that the stages are not only described conceptually but are actually encoded in a dependency graph that DVC can execute and track.

10 Model Development and Evaluation

10.1 Model Choice

The active model is a custom CNN defined in `src/models.py`. The configuration currently uses convolution channel sizes $[16, 32]$, kernel size 3, and a fully connected layer of size 128. A ResNet-18 option also exists in the repository, which shows that the codebase supports comparative experimentation rather than locking the system to one fixed model design.

10.2 Training and Evaluation Results

Training is implemented in `src/train.py`, and evaluation is handled in `src/evaluate.py`. MLflow logs parameters, metrics, and artifacts from both stages. The current generated metrics show solid performance:

Split	Accuracy	Precision	Recall	F1-score
Validation	0.9100	0.8804	0.9488	0.9134
Test	0.9219	0.9027	0.9457	0.9237

Table 1: Current validation and test performance from the repository outputs

These numbers show that the model is performing well on the held-out test set while also keeping a strong recall, which is especially relevant in a medical-imaging setting where missed positives can be costly.

11 MLflow Experiment Tracking and Model Registry

The project uses MLflow to log the models and their artifacts. Training runs log loss, F1, accuracy, precision, and recall. Parameters such as batch size, epochs, learning rate, and model choice are recorded. Artifacts such as model files and metrics folders are stored with each run. The model is also registered and promoted through MLflow's registry.

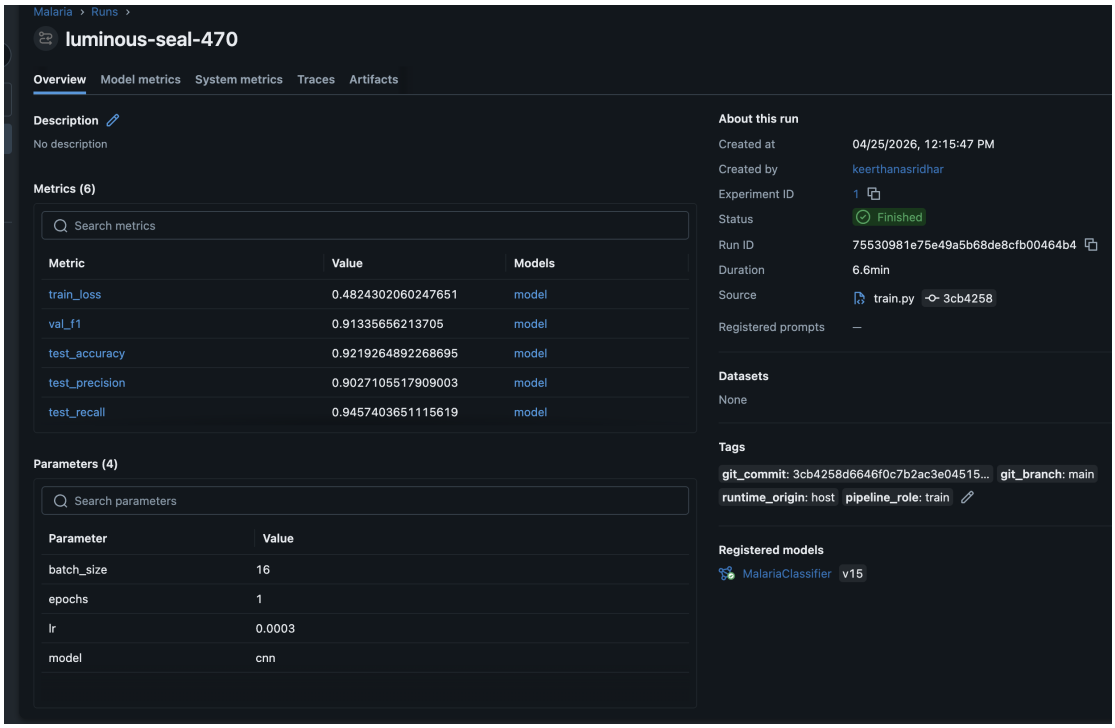


Figure 8: This MLflow run page is a strong piece of evidence because it connects code, parameters, metrics, and registry state in one place. The screenshot shows that training loss, validation F1, and test metrics were logged, while the right panel records run metadata such as the source file, Git-related tags, runtime origin, and the registered model version.

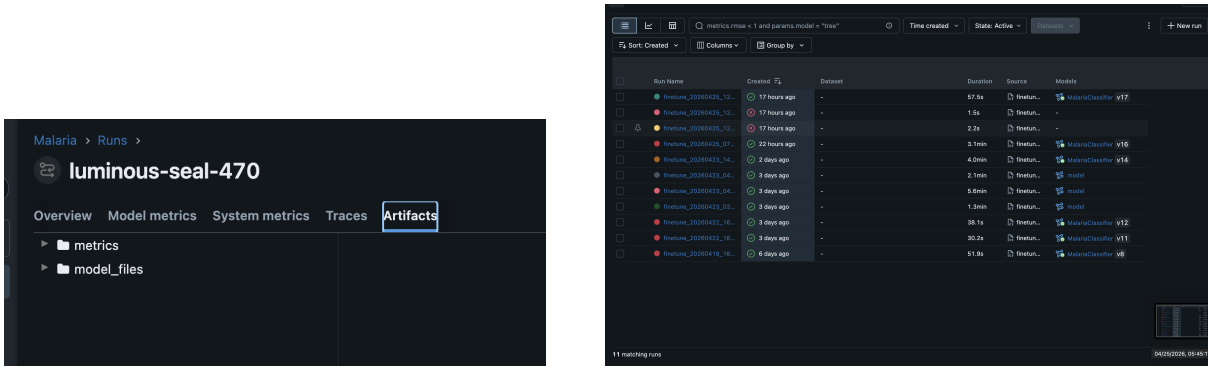


Figure 9: The left image shows the stored artifacts for a tracked run, which is important because it proves that MLflow is not only storing scalar metrics but also preserving structured outputs for later inspection. The right image shows the fine-tuning experiment history, including multiple incremental runs and promoted versions.

12 Inference, Alerts, and Monitoring

12.1 Runtime Monitoring in the UI

The frontend does not simply display predictions. It also surfaces alerts, aggregated confidence, total predictions, and drift signals. That makes the application more transparent and easier to evaluate in real time.

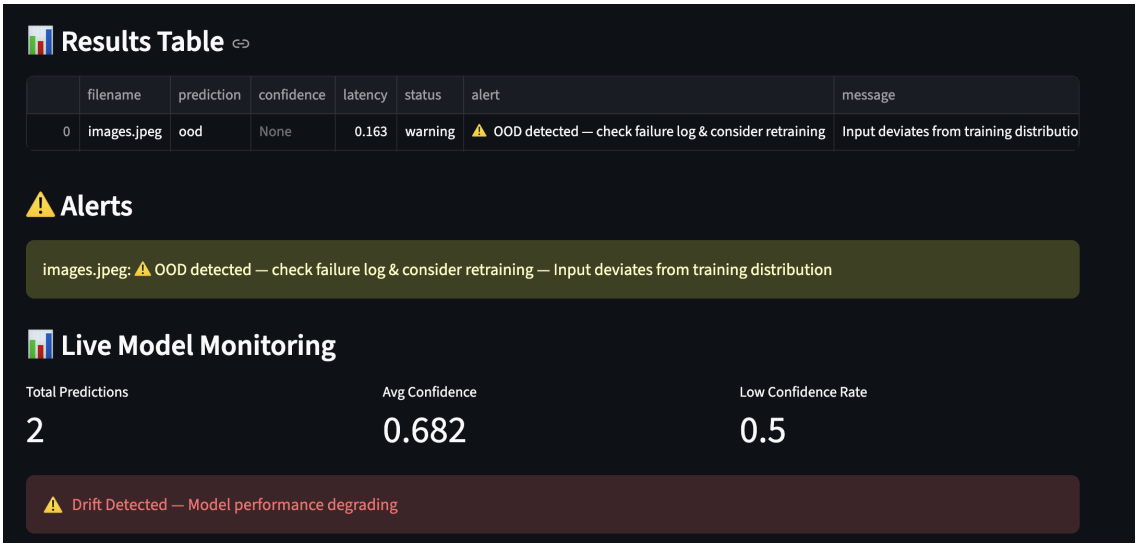


Figure 10: The results table marks the prediction as OOD, the alerts section explains that the sample deviates from the training distribution, and the live monitoring cards show the total predictions, average confidence, low-confidence rate, and drift warning.

12.2 Prometheus and Alertmanager

Prometheus scrapes backend metrics from the `/metrics` endpoint, and the alert rules watch for drift, OOD rates, high error rates, slow inference, and high invalid-input rates. When one of

these rules fires, Alertmanager groups and routes the alerts.

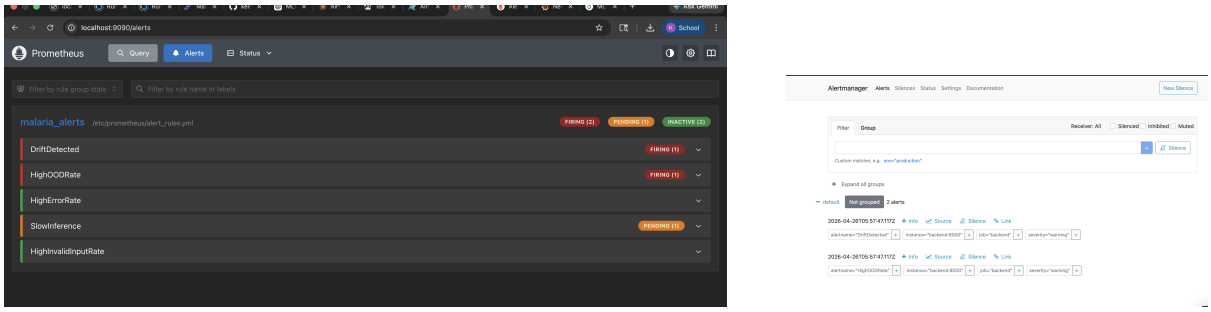


Figure 11: The Prometheus view on the left shows the alert rules in action, including DriftDetected and HighOODRate in the firing state and SlowInference in the pending state. The Alertmanager view on the right confirms that those alerts are grouped and routed downstream.

12.3 Grafana Dashboards

Grafana adds the final layer of visibility by turning raw metric streams into understandable charts and indicators.

Figure 12: The Grafana panels show two complementary views of the backend. The left screenshot focuses on system-side and confidence-related telemetry, including CPU usage and average confidence over time. The right screenshot focuses on risk signals such as OOD rate and drift status.

13 Human-in-the-Loop Feedback and Retraining

One of the more mature parts of the project is the feedback loop. When inference is uncertain or OOD, samples are stored in feedback queues. A physician or expert can review them, label them, and either place them in the labelled folder structure or upload them as a ZIP archive from the Pipeline page. After that, the Airflow DAG can process the new feedback and trigger fine-tuning.

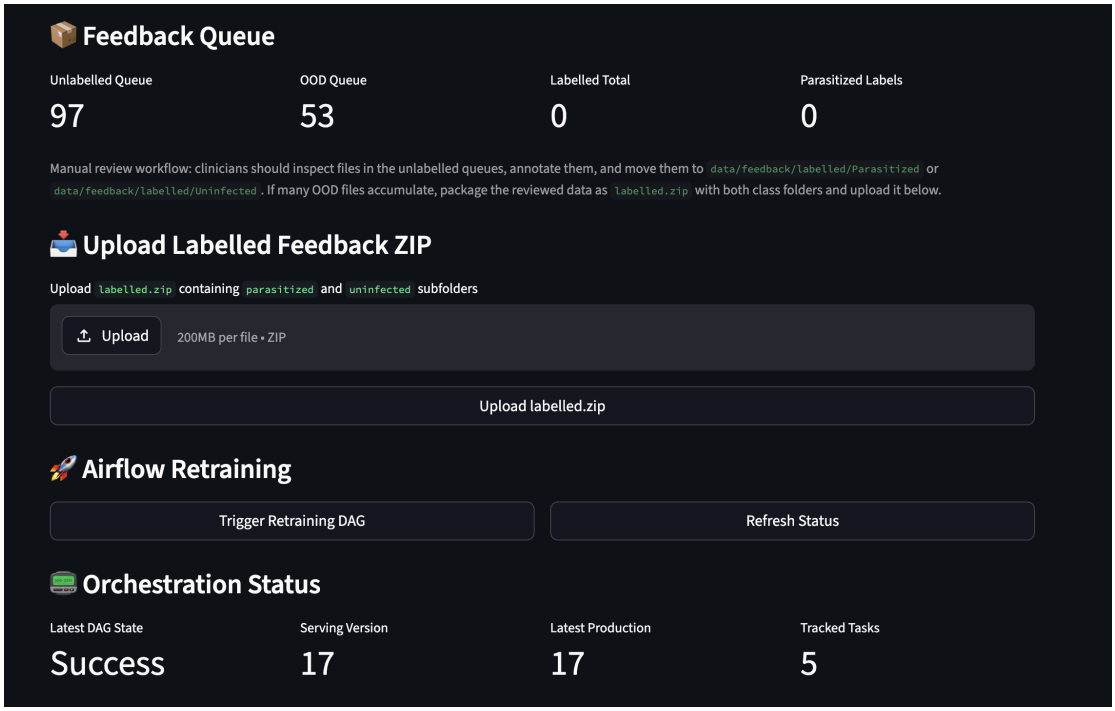
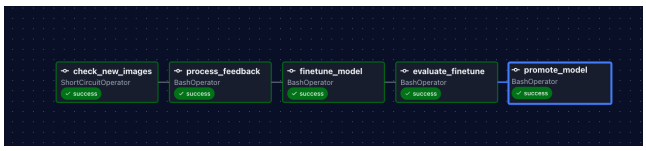


Figure 13: This screenshot shows the Pipeline page acting as an operational console rather than just a decorative dashboard. The top section exposes the current feedback queue counts, the middle section allows labelled feedback to be uploaded, and the lower section provides buttons for manual DAG triggering and status refresh. The orchestration cards at the bottom then report the latest DAG state, serving version, latest production version, and tracked task count.

The retraining pipeline itself is orchestrated through Airflow. It checks for new labelled data, processes feedback, fine-tunes the model, evaluates the candidate model, and promotes it if the validation threshold is satisfied.



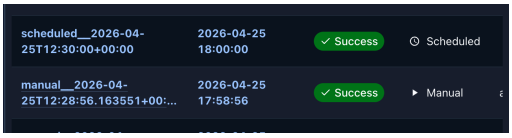


Figure 14: The Airflow screenshots show both the task structure and the operational history of the retraining DAG. The graph view on the left confirms the task sequence `check_new_images` → `process_feedback` → `finetune_model` → `evaluate_finetune` → `promote_model`. The run-history view on the right shows that both scheduled and manually triggered runs complete successfully, which is useful evidence for the evaluation rubric's emphasis on retraining orchestration and pipeline management.

The current fine-tune evaluation output reports a validation accuracy of 0.8139 against a threshold of 0.75, and the candidate was accepted. This means the feedback-driven path is not only implemented, but has also succeeded in producing a promotable model version.

14 Backend API Design

The backend in `src/app.py` exposes more than a single `/predict` route. It acts as the system's operational core. The most important endpoints are summarized below.

Endpoint	Purpose
`/health`	Liveness check for the backend service
`/ready`	Readiness check for model-serving state
`/predict`	Image inference endpoint for uploaded files
`/api/metrics`	Runtime metrics exposed to the UI and monitoring tools
`/api/failures`	Access to failure and review-log entries
`/api/feedback/stats`	Counts for unlabeled, OOD, and labelled queues
`/api/feedback/upload-labelled-zip`	Upload of physician-reviewed labelled data
`/pipeline/status`	DVC workspace state and reproducibility helper information
`/retrain`	Background trigger for the DVC reproducible workflow
`/api/retraining/trigger`	Manual trigger for the Airflow retraining DAG
`/api/retraining/status`	Latest DAG status and per-task information
`/api/model/refresh`	Manual backend refresh of served model state
`/model/info`	Metadata for the currently served model version
`/alerts`	Active alerts currently visible in the application
`/internal/alert`	Webhook receiver used by Alertmanager

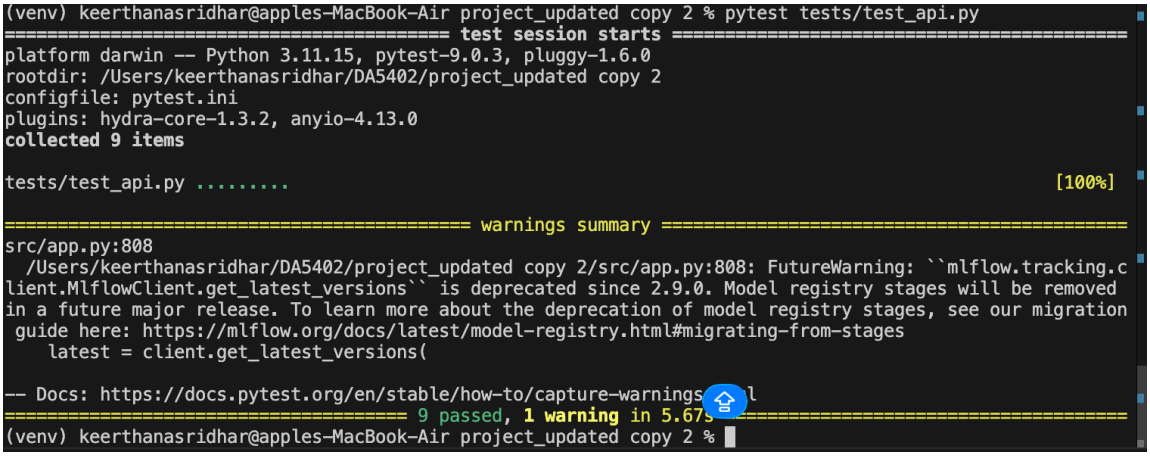
Table 2: Primary backend API surface

This API design helps satisfy the low-level design expectations in the evaluation PDF. The endpoints are not only present in documentation; they are implemented and linked to visible frontend features.

15 Testing and Software Engineering Quality

The repository contains structured design documents in `docs/`, including architecture, high-level design, low-level design, user manual, reproducibility notes, acceptance criteria, and a test plan. This gives the project a stronger engineering foundation than a repository that only contains code.

Automated testing is also present. The file `tests/test\_api.py` currently defines nine tests covering health, readiness, model info, invalid file handling, empty file handling, valid prediction, DVC status parsing, invalid-log counting, and model refresh logic.



```
(venv) keerthanasridhar@apples-MacBook-Air project_updated copy 2 % pytest tests/test_api.py
========================================= test session starts =========================================
platform darwin -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/keerthanasridhar/DA5402/project_updated copy 2
configfile: pytest.ini
plugins: hydra-core-1.3.2, anyio-4.13.0
collected 9 items

tests/test_api.py .........                                                                    [100%]

========================================== warnings summary ===========================================
src/app.py:808
  /Users/keerthanasridhar/DA5402/project_updated copy 2/src/app.py:808: FutureWarning: ``mlflow.tracking.c
lient.MlflowClient.get_latest_versions`` is deprecated since 2.9.0. Model registry stages will be removed
in a future major release. To learn more about the deprecation of model registry stages, see our migration
 guide here: https://mlflow.org/docs/latest/model-registry.html#migrating-from-stages
    latest = client.get_latest_versions(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===================================== 9 passed, 1 warning in 5.67s ====================================
(venv) keerthanasridhar@apples-MacBook-Air project_updated copy 2 %
```

Figure 15: The pytest output shows that nine backend tests pass successfully, with a single warning related to an MLflow deprecation notice rather than a functional failure.

16 Conclusion

This project shows how a medical-image classifier can be built and presented as a full MLOps application. The repository supports prediction through a user-friendly frontend, model serving through FastAPI, experiment tracking and model registry through MLflow, reproducible training through DVC, feedback-driven retraining through Airflow, and runtime monitoring through Prometheus, Grafana, and Alertmanager. The strongest aspect of the project is that these pieces are not isolated: they are connected in a workflow that allows the model to be observed, challenged, corrected, and improved over time.

The screenshots included in this report strengthen that claim. They show the UI, the prediction outputs, the feedback queue, the DVC graph, MLflow runs and artifacts, Airflow retraining, Prometheus alerts, Alertmanager routing, Grafana dashboards, Dockerized deployment, and automated tests. Together, they make the project easy to explain during viva and credible as an end-to-end MLOps submission.

A AI Disclosure

- 1) produce a boilerplate of my backend.py with the necessary basic endpoints.
- 2) produce a boilerplate of my frontend.py which is suitable for my backend.py.
- 3) how do i implement airflow retriggering via a frontend button?
- 4) mlflow sever vs mlflow local, how does the mapping happen?
- 5) can i use the files absolutely saved from within a docker work on user? if not suggest a code fix for the same as im not able to access the mlflow trains within from my local

computer

- 6) give me a script for the eda- involving pixel intensity and width and height
- 7) how does the decoupling of docker files for front and back end work?
- 8)i have a unzip script already. can you make it robust enough to handle both archive layouts and put everything into data/raw/cell$_images$
- 9) my app should still work even if mlflow has nothing in it. can you make sure the backend falls back to a local checkpoint cleanly 9) how can i promote my model to production via code? give me a sample
- 10) what are the good test cases for this project? give me a couple of basic test cases
- 11) generate a sample script tat lets me push my data from labelled folder to unlabelled folder
- 12) how can i generate a button, that lets the user upload a labelled.zip, that should dynamically check for (parasitized and uninfected subfolders) and extract those to data/feedback/labelled
- 13) this mlflow server should be started whenever we run the app? explain again?
- 14) give me a section in my backend that can invalidate corrupt or invalid images
- 15) how can i keep the metrics as dynamic and refreshable?
- 16) give me some guidance on how to log metrics

The rest of the prompts used were for troubleshooting errors and bugs in the code. AI was also used to generate the readme and the report template.